

Lecture 18

 PDF

I want you to create a comprehensive summary and group the concepts and ideas into sections.

The motivation for attention comes from the limitations of Recurrent Neural Networks (RNNs) and encoder-decoder architectures, which struggle with a "bottleneck" problem—forcing all information from an input sentence into a single fixed-size vector. 🧠 🧠

思考型 

The screenshot shows the Gemini AI web interface. At the top left is the Gemini logo. The main content area displays a structured explanation of Transformer architecture components. It starts with "Key Components" which includes Residual Connections, Layer Normalization (Add & Norm), and Embeddings. This is followed by section "3. Position Encodings", which explains why position information is needed since Transformers process tokens in parallel. It details three methods: Sinusoidal (Original), Learned Relative Encoding, and RoPE (Rotary Position Embedding). A text input box at the bottom contains the Chinese prompt "來和我寫作或開發吧！". The interface also features a sidebar with icons for menu, edit, and settings, and a top right corner with a PRO badge and a user profile icon.

Gemini

PRO

### 5. Efficiency and Scaling: Mixture of Experts (MoE)

As models get larger, the computational cost becomes prohibitive. MoE is a solution to scale up parameters without exploding compute costs.

- **The Concept:** Instead of one massive Feed-Forward Network (FFN) that every token must pass through, the model has many smaller "expert" networks. ⓘ ⓘ
- **Routing:** A "Router" decides which specific expert(s) should process a given token. For example, in the "Switch Transformer," a router selects specific FFN experts based on a probability calculation. ⓘ ⓘ
- **Sparse Activation:**
  - **Total Parameters:** Very high (e.g., OLMoE-1B-7B has 6.9B total params).
  - **Active Parameters:** Very low (e.g., only 1.3B are used per token). This makes inference much faster than a dense model of the same total size. ⓘ ⓘ
- **Modern Examples:** Models like DeepSeek V3 and Qwen use fine-grained experts (e.g., 64 small experts, activating 8 at a time) or shared experts to optimize performance. ⓘ ⓘ

#### Next Step

Would you like me to generate a set of practice questions or a "cheat sheet" comparing the mathematical differences between **RoPE** and **Standard Position Encoding** to help you prepare for an exam?

+ 🗨️ 🖼️ Canvas ✕

思考型 ▾ 🔊

Gemini 可能會出錯，請查證。

I would say overall it summarizes the lecture notes really well



Cite all the papers that are used in these concepts sections.

Based on the lecture notes provided, here are the seminal papers that correspond to the concepts and architectures discussed. I have grouped them by the sections from our summary so you can map the theory directly to the source literature.

The lectures introduce the fundamental architecture that moved away from RNNs to pure attention mechanisms.

- **The Paper:** "*Attention Is All You Need*" (Vaswani et al., 2017).
- **Where it appears in the notes:** The lectures explicitly use this title to introduce the architecture. This paper introduced the Encoder-Decoder structure, the concept of Multi-Head Attention, and the original sinusoidal positional encodings.

來和我寫作或開發吧！

+  Canvas X

思考型 ▾



Gemini 可能會出錯，請查證。



I am impressed that it is able to find all those papers. however. I would love it to tell me where in the paper are they referring to.

The lectures focus heavily on models designed for autoregressive tasks (predicting the next token), specifically the GPT family.

- **The Paper:** "*Language Models are Few-Shot Learners*" (Brown et al., 2020) — This is the GPT-3 paper.
- **Where it appears in the notes:** The lectures detail the GPT-3 architecture, emphasizing its decoder-only nature and its massive scale (96 layers, 175 billion parameters).
- **Key Concept:** This paper demonstrated that scaling up the Transformer decoder allows for "few-shot" learning without fine-tuning.

The lectures contrast GPT-3 with models designed to learn bidirectional context for tasks like classification.

- **The Paper:** "BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding" (Devlin et al., 2018).
- **Where it appears in the notes:** The lectures discuss BERT explicitly, defining it as "Bidirectional Encoder Representations from Transformers".
- **Key Concept:** It introduced the "Masked Language Model" (MLM) training objective (predicting `<mask>` tokens), which is distinct from the left-to-right generation of GPT.

來和我寫作或開發吧！

+  Canvas x

思考型 ▾



Gemini 可能會出錯，請查證。



#### 4. Positional Embeddings

The lectures discuss the evolution from fixed encodings to learned rotations.

- **The Paper:** "*RoFormer: Enhanced Transformer with Rotary Position Embedding*" (Su et al., 2021).
- **Where it appears in the notes:** This corresponds to the section on **RoPE**. The lectures explain how this method modifies the attention calculation using a rotation matrix to capture relative positions.

## 5. Mixture of Experts (MoE) & Efficiency

The lectures cover methods to scale parameters without exploding computational cost.

- **The Paper:** "Switch Transformers: Scaling to Trillion Parameter Models with Simple and Efficient Sparsity" (Fedus et al., 2021).
- **Where it appears in the notes:** The lectures provide a specific diagram for the **Switch Transformer** and explicitly cite "Fedus et al 2021". This introduced the idea of replacing dense feed-forward layers with sparse "experts" routed by a gating mechanism.
- **The Paper:** "OLMoE: Open Mixture-of-Experts Language Models" (Muennighoff et al., 2024).
- **Where it appears in the notes:** The lectures discuss **OLMoE** and compare its architecture (using small, fine-grained experts) to dense models like Llama.

## 6. New Architectures (Beyond Transformers)

來和我寫作或開發吧！



Canvas X

思考型 ▾



Gemini 可能會出錯，請查證。



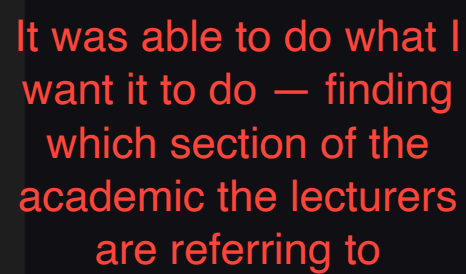
## 6. New Architectures (Beyond Transformers)

The lectures briefly touch upon alternatives to the standard attention mechanism.

- **The Paper:** "Mamba: Linear-Time Sequence Modeling with Selective State Spaces" (Gu & Dao, 2023).
- **Where it appears in the notes:** The lectures mention **Mamba** and **SSM** (State Space Models) alongside diagrams of **H3** layers. These models aim to be more efficient than Transformers for very long sequences.

Would you like me to create a "Reading Guide" for one of these specific papers (e.g., Attention Is All You Need) that highlights exactly which sections you should focus on to pass your class?

I love how you found those papers. I also like how you tell me where in teh lecture do they refer to these lectures. But can you also tell me where in the paper (for each of the paper you pointed out) are they pointing at and referring to? Paper is long and it would take forever for me to read it. would be great if you can tell me exactly which sections of the paper you are referring to



Here it generated a graph. Not sure what the graph is for. It doesn't explain either section 3.1 3.2, 3.5. This graph is also too simple

Here is exactly where you should look in each paper to find the concepts discussed in your lectures.

**Lecture Context:** Your notes cover the "Add & Norm" structure, Multi-Head Attention, and the specific equation for Scaled Dot-Product Attention.  

- Go to Section 3: Model Architecture.
  - Read Section 3.1: This explains the **Encoder and Decoder** stacks depicted in your lecture slides. 
  - Read Section 3.2 (Crucial): This contains the famous equation  $Attention(Q, K, V)$  and the diagram for **Scaled Dot-Product Attention** that appears in your notes. 
  - Read Section 3.5: This explains the **Positional Encoding** (sinusoidal version) mentioned in Lecture 19.  



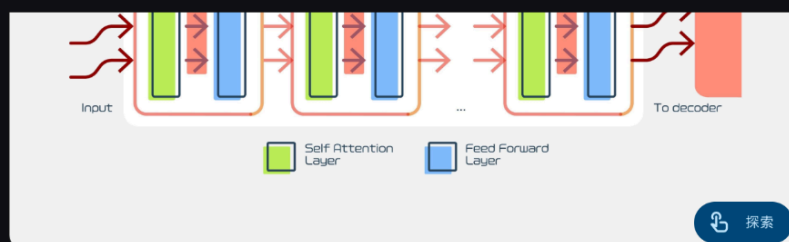
來和我寫作或開發吧！

+ Canvas X

思考型 ▼



Gemini 可能會出錯，請查證。



**Lecture Context:** The notes focus on the "Decoder-only" architecture, the massive parameter count (175B), and the autoregressive nature of the model.  

- **Go to Section 2: Approach.**
  - **Read Section 2.1 (Model and Architectures):** This is short but vital. It explicitly states they use the same architecture as GPT-2 (decoder-only).
  - **Look at Table 2.1:** Your lecture mentions "96 layers" and "96 heads". This specific table in the paper lists those exact hyperparameters for the model size "GPT-3 175B".

**Lecture Context:** The notes discuss the "Encoder-only" structure, the masking of tokens (`<mask>`), and the use of the `<cls>` token for classification.   

- Go to Section 3: BERT.

來和我寫作或開發吧！

+  Canvas X

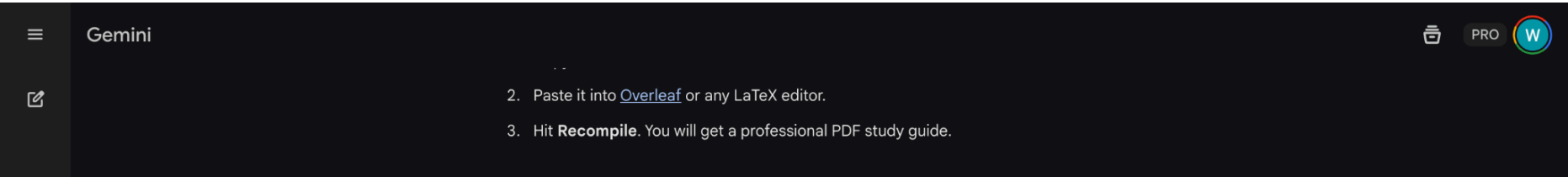
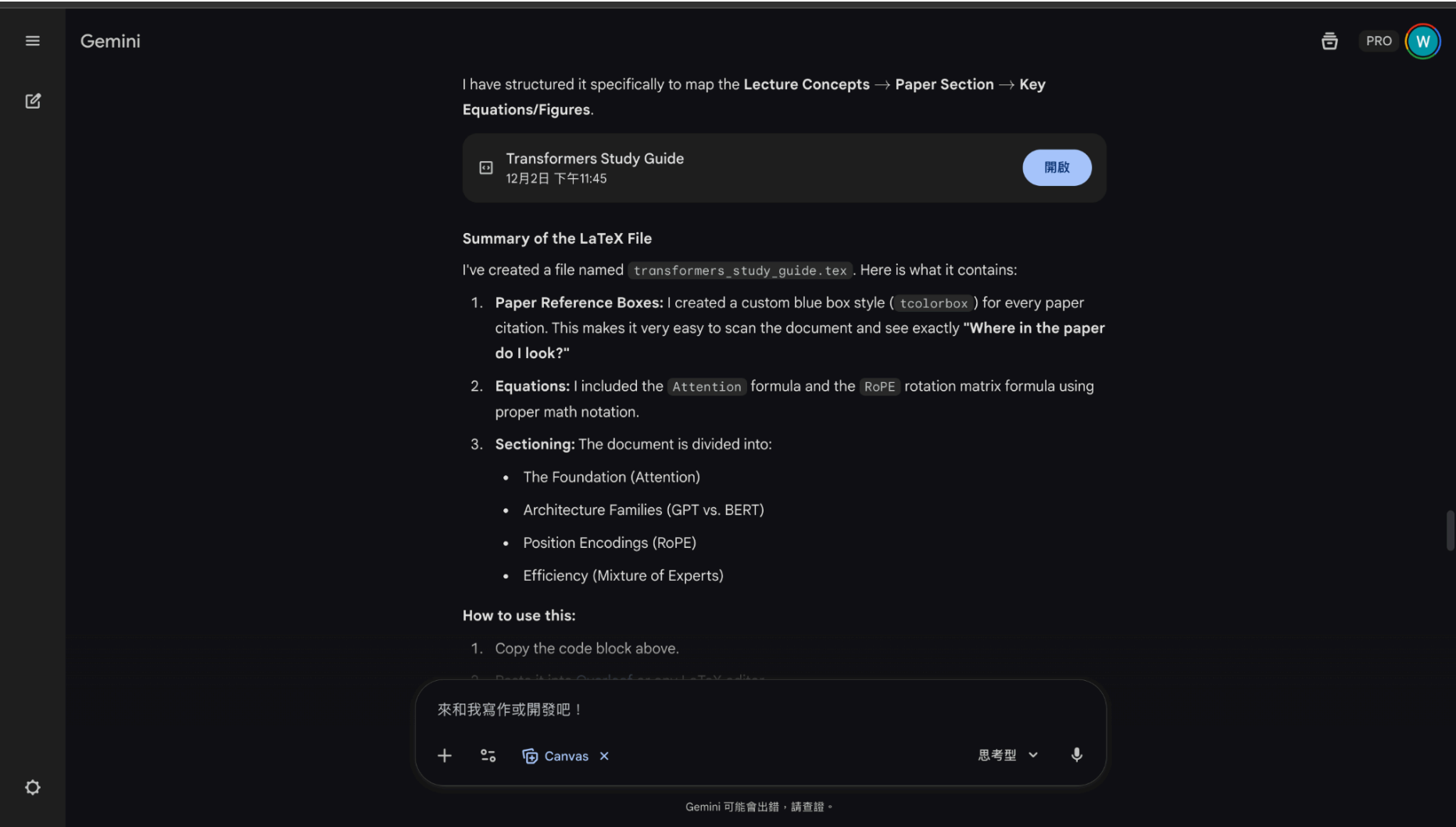
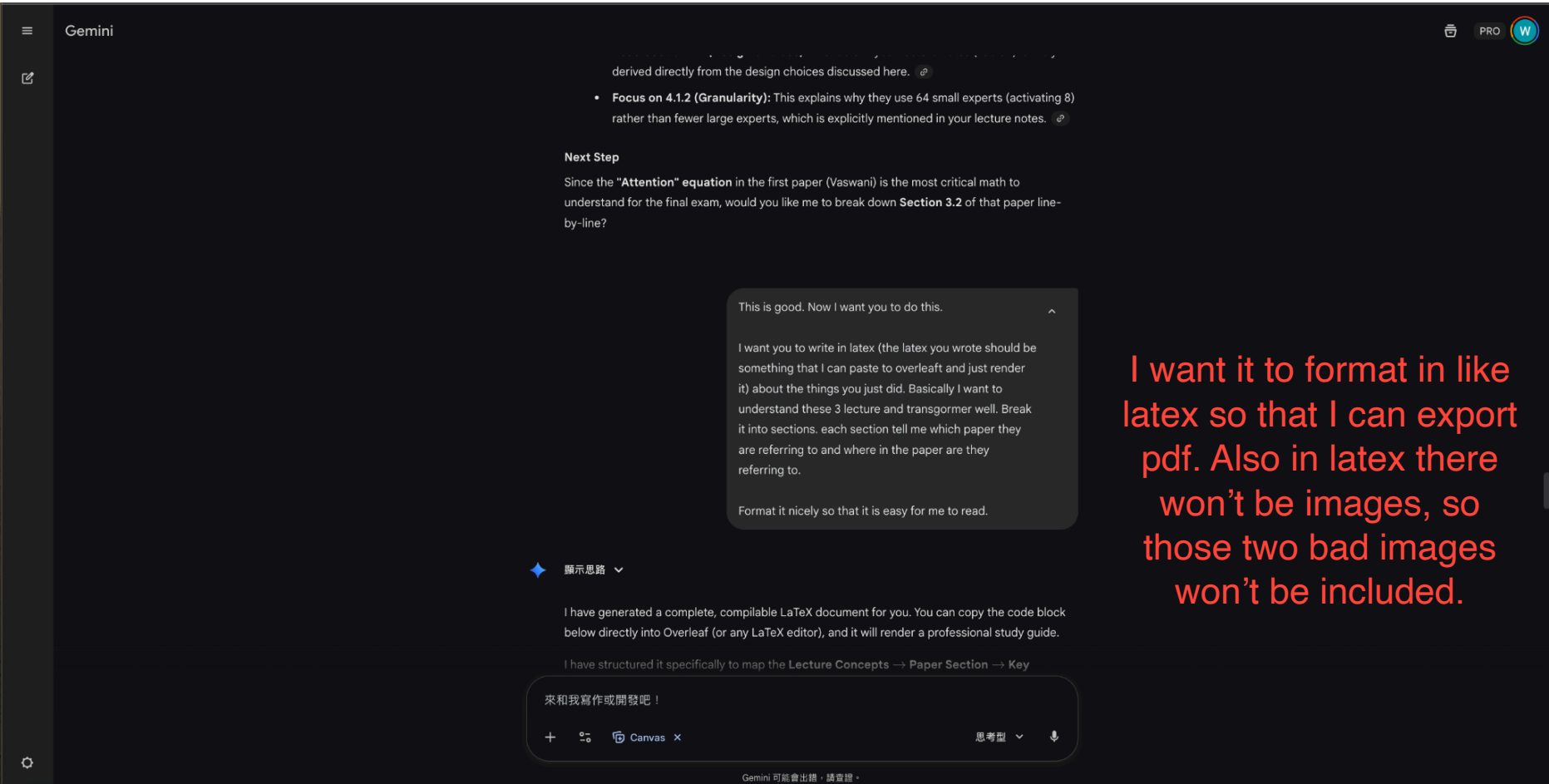
思考型 ▾



Gemini 可能會出錯，請查證。









# Deep Learning: Transformers & Advanced Architectures

Lecture Notes 18, 19, & 20 Summary

Deep Learning Class Study Guide

December 3, 2025

## 1 Introduction

This document synthesizes concepts from Lectures 18, 19, and 20. It maps theoretical concepts directly to the seminal papers that introduced them, specifying exactly which sections of those papers to consult.

## 2 The Foundation: Attention & The Transformer

*Lecture 18 & 19*

The Transformer moves away from Recurrent Neural Networks (RNNs) to a purely attention-based mechanism. It solves the "bottleneck" problem of RNNs by allowing the decoder to look back at the entire input sequence simultaneously.

### 2.1 Scaled Dot-Product Attention

The core mathematical operation. Instead of a single hidden state, we have three vectors for every token:

- **Query ( $Q$ ):** What I am looking for.
- **Key ( $K$ ):** What defines the data I have.
- **Value ( $V$ ):** The actual content of the data.

The attention score is calculated by the dot product of the Query and Key. It is scaled by  $\sqrt{d_k}$  to prevent vanishing gradients in the softmax.

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) V \quad (1)$$

#### Reference: Attention Is All You Need (Vaswani et al., 2017)

- **Go to:** Section 3.2 (Attention).
- **Look for:** Equation 1. This is the exact formula used in the lecture slides.
- **Key Insight:** Figure 2 in the paper visualizes the "Scaled Dot-Product Attention" block exactly as drawn in your notes.

I love how they format it  
very nicely and have the  
reference listed in a blue  
box

## 2.2 Multi-Head Attention

To capture different types of relationships (e.g., grammatical vs. semantic) simultaneously, the model runs the attention mechanism multiple times in parallel ("heads") and concatenates the results.

### Reference: Attention Is All You Need (Vaswani et al., 2017)

- **Go to:** Section 3.2.2 (Multi-Head Attention).
- **Look for:** Equation 2, which defines how the heads are concatenated ( $\text{Concat}(\text{head}_1, \dots, \text{head}_h)W^O$ ).

## 3 Architecture Families: Encoder vs. Decoder

Lecture 19 & 20

The original Transformer had both an Encoder and a Decoder. Modern architectures usually split these.

### 3.1 Decoder-Only (Generative Models)

Models like **GPT-3** are designed to predict the next token. They use **Masked Self-Attention** to prevent the model from seeing the future words during training.

- **Task:** Autoregressive Generation.
- **Constraint:** Token at position  $t$  can only attend to positions 1 to  $t - 1$ .

### Reference: Language Models are Few-Shot Learners (Brown et al., 2020)

- **Go to:** Section 2.1 (Model and Architectures).
- **Look for:** The text explicitly states they use the same model and architecture as GPT-2 (which is a decoder-only transformer).
- **Hyperparameters:** See **Table 2.1** in the paper to find the "96 layers" and "96 heads" mentioned in your lecture notes for the 175B parameter model.

### 3.2 Encoder-Only (Understanding Models)

Models like **BERT** are designed to understand the full context of a sentence (left and right).

- **Task:** Masked Language Modeling (MLM). Randomly hide 15% of tokens and predict them.
- **Constraint:** Must process the whole sentence at once; cannot be used easily for text generation.

### Reference: BERT: Pre-training of Deep Bidirectional Transformers (Devlin et al., 2018)

- **Go to:** Section 3.1 (Pre-training BERT).
- **Look for:** The subsection titled **Task #1: Masked LM**. This explains the "15% masking" rule found in your notes.
- **Key Visual:** Figure 1 in the paper compares the architecture of BERT vs. GPT.

## 4 Position Encodings: RoPE

### Lecture 20

Since self-attention is permutation invariant (it treats "dog bites man" the same as "man bites dog" without extra help), we need to inject position information. The modern standard is **Rotary Position Embedding (RoPE)**.

Instead of adding a vector, RoPE *rotates* the Query and Key vectors in the geometric space. The angle of rotation depends on the position in the sequence.

$$R_{\theta,m}x = \begin{pmatrix} \cos m\theta & -\sin m\theta \\ \sin m\theta & \cos m\theta \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \end{pmatrix} \quad (2)$$

**Reference: RoFormer: Enhanced Transformer with Rotary Position Embedding (Su et al., 2021)**

- **Go to:** Section 3.2.1 (2D Word Vectors).
- **Look for:** Equation 13 or 14. This defines the rotation matrix containing cos and sin that appears in your Lecture 20 slides.
- **Why it matters:** It proves that  $\langle f_q(x_m, m), f_k(x_n, n) \rangle = g(x_m, x_n, m - n)$ , meaning the attention score depends only on the *relative* distance ( $m - n$ ).

## 5 Efficiency: Mixture of Experts (MoE)

### Lecture 20

As models scale to trillions of parameters, the computational cost becomes too high. MoE replaces the dense Feed-Forward Network (FFN) with many sparse "Experts".

### 5.1 Routing

A "Router" decides which tokens go to which expert. For example, a noun might go to Expert A, while a verb goes to Expert B.

- **Sparse Activation:** A model might have 7B parameters, but only use 1B for any single token.

**Reference: Switch Transformers (Fedus et al., 2021)**

- **Go to:** Section 2.1 (Simplifying the Router).
- **Look for:** Figure 2 in the paper. It mirrors the diagram in your lecture notes showing tokens being "switched" to different FFN blocks.

**Reference: OLMoE: Open Mixture-of-Experts Language Models (Muennighoff et al., 2024)**

- **Go to:** Section 4.1 (Design Choices).
- **Look for:** The discussion on "Granularity" (Section 4.1.2). This explains the trade-off mentioned in your notes about using many small experts (64 total, 8 active) versus fewer large ones.

Overall, I really like the format. However, I feel like it is not detail enough (I would ask it to make it more detail later)



This is great.



Now I want you to create a new section call further work (one way to learn the topic better is to see what the state of the art of that topic is, and most importantly how it evolved over the time to that state of the art, how each method improve over the past one)



顯示思路

I have updated the document to include a "Further Work: State of the Art & Evolution" section.

This new section maps the evolution from the standard Transformer (2017) to today's architectures (Llama 3, Mamba, etc.), focusing on **how** and **why** specific components changed (e.g., why we moved from ReLU to SwiGLU, or LayerNorm to RMSNorm).

Here is the updated, complete LaTeX file:

來和我寫作或開發吧！



Canvas



思考型



Gemini 可能會出錯，請查證。

# Deep Learning: Transformers & Advanced Architectures

Lecture Notes 18, 19, & 20 Summary

Deep Learning Class Study Guide

December 3, 2025

## 1 Introduction

This document synthesizes concepts from Lectures 18, 19, and 20. It maps theoretical concepts directly to the seminal papers that introduced them, specifying exactly which sections of those papers to consult.

## 2 The Foundation: Attention & The Transformer

*Lecture 18 & 19*

The Transformer moves away from Recurrent Neural Networks (RNNs) to a purely attention-based mechanism. It solves the "bottleneck" problem of RNNs by allowing the decoder to look back at the entire input sequence simultaneously.

### 2.1 Scaled Dot-Product Attention

The core mathematical operation. Instead of a single hidden state, we have three vectors for every token:

- **Query ( $Q$ ):** What I am looking for.
- **Key ( $K$ ):** What defines the data I have.
- **Value ( $V$ ):** The actual content of the data.

The attention score is calculated by the dot product of the Query and Key. It is scaled by  $\sqrt{d_k}$  to prevent vanishing gradients in the softmax.

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) V \quad (1)$$

#### Reference: Attention Is All You Need (Vaswani et al., 2017)

- **Go to:** Section 3.2 (Attention).
- **Look for:** Equation 1. This is the exact formula used in the lecture slides.
- **Key Insight:** Figure 2 in the paper visualizes the "Scaled Dot-Product Attention" block exactly as drawn in your notes.

## 2.2 Multi-Head Attention

To capture different types of relationships (e.g., grammatical vs. semantic) simultaneously, the model runs the attention mechanism multiple times in parallel ("heads") and concatenates the results.

### Reference: Attention Is All You Need (Vaswani et al., 2017)

- **Go to:** Section 3.2.2 (Multi-Head Attention).
- **Look for:** Equation 2, which defines how the heads are concatenated ( $\text{Concat}(\text{head}_1, \dots, \text{head}_h)W^O$ ).

## 3 Architecture Families: Encoder vs. Decoder

Lecture 19 & 20

The original Transformer had both an Encoder and a Decoder. Modern architectures usually split these.

### 3.1 Decoder-Only (Generative Models)

Models like **GPT-3** are designed to predict the next token. They use **Masked Self-Attention** to prevent the model from seeing the future words during training.

- **Task:** Autoregressive Generation.
- **Constraint:** Token at position  $t$  can only attend to positions 1 to  $t - 1$ .

### Reference: Language Models are Few-Shot Learners (Brown et al., 2020)

- **Go to:** Section 2.1 (Model and Architectures).
- **Look for:** The text explicitly states they use the same model and architecture as GPT-2 (which is a decoder-only transformer).
- **Hyperparameters:** See **Table 2.1** in the paper to find the "96 layers" and "96 heads" mentioned in your lecture notes for the 175B parameter model.

### 3.2 Encoder-Only (Understanding Models)

Models like **BERT** are designed to understand the full context of a sentence (left and right).

- **Task:** Masked Language Modeling (MLM). Randomly hide 15% of tokens and predict them.
- **Constraint:** Must process the whole sentence at once; cannot be used easily for text generation.

### Reference: BERT: Pre-training of Deep Bidirectional Transformers (Devlin et al., 2018)

- **Go to:** Section 3.1 (Pre-training BERT).
- **Look for:** The subsection titled **Task #1: Masked LM**. This explains the "15% masking" rule found in your notes.
- **Key Visual:** Figure 1 in the paper compares the architecture of BERT vs. GPT.



## 4 Position Encodings: RoPE

### Lecture 20

Since self-attention is permutation invariant (it treats "dog bites man" the same as "man bites dog" without extra help), we need to inject position information. The modern standard is **Rotary Position Embedding (RoPE)**.

Instead of adding a vector, RoPE *rotates* the Query and Key vectors in the geometric space. The angle of rotation depends on the position in the sequence.

$$R_{\theta,m}x = \begin{pmatrix} \cos m\theta & -\sin m\theta \\ \sin m\theta & \cos m\theta \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \end{pmatrix} \quad (2)$$

**Reference: RoFormer: Enhanced Transformer with Rotary Position Embedding (Su et al., 2021)**

- **Go to:** Section 3.2.1 (2D Word Vectors).
- **Look for:** Equation 13 or 14. This defines the rotation matrix containing cos and sin that appears in your Lecture 20 slides.
- **Why it matters:** It proves that  $\langle f_q(x_m, m), f_k(x_n, n) \rangle = g(x_m, x_n, m - n)$ , meaning the attention score depends only on the *relative* distance ( $m - n$ ).

## 5 Efficiency: Mixture of Experts (MoE)

### Lecture 20

As models scale to trillions of parameters, the computational cost becomes too high. MoE replaces the dense Feed-Forward Network (FFN) with many sparse "Experts".

### 5.1 Routing

A "Router" decides which tokens go to which expert. For example, a noun might go to Expert A, while a verb goes to Expert B.

- **Sparse Activation:** A model might have 7B parameters, but only use 1B for any single token.

**Reference: Switch Transformers (Fedus et al., 2021)**

- **Go to:** Section 2.1 (Simplifying the Router).
- **Look for:** Figure 2 in the paper. It mirrors the diagram in your lecture notes showing tokens being "switched" to different FFN blocks.

**Reference: OLMoE: Open Mixture-of-Experts Language Models (Muennighoff et al., 2024)**

- **Go to:** Section 4.1 (Design Choices).
- **Look for:** The discussion on "Granularity" (Section 4.1.2). This explains the trade-off mentioned in your notes about using many small experts (64 total, 8 active) versus fewer large ones.

## 6 Further Work: State of the Art & Evolution

*Lecture 20 (DeepSeek, Mamba, Llama)*

To understand where the field is going, we must look at how the standard Transformer components are being evolved or replaced to handle efficiency and longer contexts.

### 6.1 Beyond $O(N^2)$ : Linear-Time Sequence Modeling

The standard Transformer has quadratic complexity ( $O(N^2)$ ) due to the attention matrix. New architectures like **Mamba** (State Space Models) achieve linear scaling ( $O(N)$ ) while maintaining high performance.

#### Evolution:

- **RNNs:** Linear time, but forget context easily (Vanishing Gradient).
- **Transformers:** Perfect memory, but quadratic cost (slow for long texts).
- **Mamba/SSM:** Linear time like RNNs, but with better long-term memory properties.

#### Reference: Mamba: Linear-Time Sequence Modeling (Gu & Dao, 2023)

- **Context:** Lecture 20, Page 10 (Diagrams of H3 and Mamba).
- **Key Idea:** Replacing the Attention mechanism with a selective state space model that compresses context efficiently.

### 6.2 Component Refinements: SwiGLU & RMSNorm

Modern "Llama-style" architectures differ significantly from the original 2017 Transformer in their sub-components.

#### Activation Function Evolution:

- **ReLU (Original):**  $\max(0, x)$ . Simple but can result in "dead neurons".
- **GELU (GPT-3):** Smoother approximation of ReLU.
- **SwiGLU (PaLM/Llama):** A Gated Linear Unit using the Swish activation. It adds a "gate" to the feed-forward network, allowing it to control information flow more precisely.

#### Reference: GLU Variants Improve Transformer (Shazeer, 2020)

- **Context:** Lecture 20, Page 10 (Equations for GLU/SwiGLU).
- **Significance:** This paper empirically showed that Gated Linear Units offer better performance for compute than standard FFNs.

#### Normalization Evolution:

- **LayerNorm (Original):** Centers and scales the input.
- **RMSNorm (Llama/Gemma):** Root Mean Square Normalization. It skips the mean-centering step, making it computationally cheaper and often more stable.

#### Reference: Root Mean Square Layer Normalization (Zhang & Sennrich, 2019)

- **Context:** Lecture 19, Page 5 (Discussion on RMS norm input).

In the prompt, I asked how they evolve over time, but here it doesn't really give me the time (like the year). I am still happy about this though.



I feel like your study guide is too simple. In the lecture notes we have a lot of math equations and stuff like that. Can you include that

顯示思路

I have updated the study guide to include the rigorous mathematical formulations found in your lecture notes.

I added:

1. **Matrix Projections:** The explicit linear algebra for creating Queries, Keys, and Values from the input  $X$  (Lecture 18).
2. **RoPE Derivation:** The detailed rotation matrix algebra showing how relative position is preserved (Lecture 20).
3. **Modern Activations:** The exact equations for **SwiGLU** and **Gated Linear Units** compared to standard ReLU (Lecture 20).
4. **RMSNorm:** The specific formulation used in Llama models (Lecture 19/20).

Here is the updated, math-heavy LaTeX file:

來和我寫作或開發吧！

+ Canvas x

思考型 ▾



# Deep Learning: Transformers & Advanced Architectures

Lecture Notes 18, 19, & 20 Summary (Advanced Math Edition)

Deep Learning Class Study Guide

December 3, 2025

## 1 Introduction

This document synthesizes concepts from Lectures 18, 19, and 20. It maps theoretical concepts directly to the seminal papers that introduced them and includes the rigorous mathematical formulations required for the exam.

## 2 The Foundation: Attention & The Transformer

*Lecture 18 & 19*

The Transformer relies on projecting inputs into specific subspaces to calculate compatibility.

### 2.1 Linear Projections (Matrix Form)

Given an input matrix  $X \in \mathbb{R}^{N \times d_{model}}$  (where  $N$  is sequence length), we do not use  $X$  directly. We learn three weight matrices  $W^Q, W^K, W^V$  to project  $X$ :

$$Q = XW^Q \in \mathbb{R}^{N \times d_k} \quad (1)$$

$$K = XW^K \in \mathbb{R}^{N \times d_k} \quad (2)$$

$$V = XW^V \in \mathbb{R}^{N \times d_v} \quad (3)$$

#### Reference: Lecture 18, Page 9

The lecture notes define the projection explicitly as:

$$Q = \beta_q \mathbf{1}^T + \Omega_q X$$

Where  $\Omega_q$  represents the weight matrix and  $\beta_q$  the bias term.

### 2.2 Scaled Dot-Product Attention

The attention score  $A$  is a row-stochastic matrix (rows sum to 1) representing the similarity between every pair of tokens.

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) V \quad (4)$$

The scalar formulation for a single query vector  $\vec{q}_n$  attending to all keys  $\vec{k}_m$ :

$$\alpha(\vec{q}_n, \vec{k}_m) = \frac{\exp(\langle \vec{q}_n, \vec{k}_m \rangle / \sqrt{d_k})}{\sum_{j=1}^N \exp(\langle \vec{q}_n, \vec{k}_j \rangle / \sqrt{d_k})} \quad (5)$$

#### Reference: Attention Is All You Need (Vaswani et al., 2017)

- **Go to:** Section 3.2.
- **Scaling Factor:** The division by  $\sqrt{d_k}$  is crucial. Without it, for large  $d_k$ , the dot products grow large in magnitude, pushing the softmax function into regions where it has extremely small gradients (vanishing gradients).

## 3 Position Encodings: RoPE (Rotary Position Embedding)

### Lecture 20

Standard absolute position encodings are added:  $x_{pos} = x_{emb} + p_{pos}$ . RoPE *multiplies* the context vector by a rotation matrix.

### 3.1 The Rotation Matrix

For a 2-dimensional subspace, the rotation of a vector  $\vec{x}$  by position  $m$  is defined as:

$$f(\vec{x}, m) = R_{\Theta, m} \vec{x} = \begin{pmatrix} \cos m\theta & -\sin m\theta \\ \sin m\theta & \cos m\theta \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \end{pmatrix} \quad (6)$$

### 3.2 Relative Position Property

The core mathematical benefit is that the dot product between two rotated vectors depends **only** on their relative distance ( $m - n$ ), not their absolute positions.

$$\langle f_q(\vec{q}, m), f_k(\vec{k}, n) \rangle = \vec{q}^T R_m^T R_n \vec{k} = \vec{q}^T R_{n-m} \vec{k} \quad (7)$$

Since  $R_m^T R_n = R_{n-m}$ , the attention score is translation invariant.

#### Reference: RoFormer (Su et al., 2021) / Lecture 20 Page 2

The lecture notes explicitly write:

$$\langle \vec{q}, \vec{k} \rangle = \|\vec{q}\| \cdot \|\vec{k}\| \cdot \cos(\theta_{relative})$$

This confirms the score relies purely on the angle difference introduced by the rotation.

## 4 Modern Components: Gated Architectures

### Lecture 20

The standard Feed-Forward Network (FFN) in the original Transformer was:

$$\text{FFN}(x) = \text{ReLU}(xW_1 + b_1)W_2 + b_2$$

Modern Large Language Models (LLMs) like Llama and PaLM use **SwiGLU**.

## 4.1 GLU and SwiGLU Equations

A Gated Linear Unit (GLU) acts as a logical gate, controlling information flow via element-wise multiplication ( $\odot$ ).

**Standard GLU:**

$$\text{GLU}(x) = (xW + b) \odot \sigma(xV + c) \quad (8)$$

Where  $\sigma$  is the sigmoid function.

**SwiGLU (Used in Llama/PaLM):** Replaces sigmoid with the Swish activation function ( $\text{Swish}(x) = x \cdot \sigma(x)$ ).

$$\text{SwiGLU}(x) = (xW + b) \odot \text{Swish}(xV + c) \quad (9)$$

### Reference: GLU Variants Improve Transformer (Shazeer, 2020)

- **Lecture Ref:** Lecture 20, Page 10.
- **Why?** It adds learnable gating parameters ( $V, c$ ) allowing the network to select which features are preserved, yielding better performance for the same compute budget.

## 5 Normalization: RMSNorm

Lecture 19

Original Transformers used LayerNorm, which subtracts the mean and divides by the variance. Modern architectures use RMSNorm (Root Mean Square Normalization), which assumes the mean is zero, saving computation.

$$\text{RMSNorm}(x) = \frac{x}{\sqrt{\frac{1}{d} \sum_{i=1}^d x_i^2 + \epsilon}} \cdot \gamma \quad (10)$$

Where  $\gamma$  is a learnable scaling parameter.

### Reference: Lecture 19, Page 5

The notes discuss input normalization:

$$\text{RMS-norm} = \frac{1}{\sqrt{32,000}}$$

This implies ensuring the input variance is controlled before entering the attention layers.

Fantastic, now it is much more detailed. But then their whole “Further Work” just completely vanishes

Gemini

PRO

W

wait where did you Further work section go???

✦ 顯示思路

Apologies! In focusing on the advanced math equations for SwiGLU and RMSNorm, I accidentally overwrote the **Mixture of Experts (MoE)** and **Mamba (State Space Models)** sections.

I have restored them and combined everything into one "Master" document. This version now contains **all** the advanced math (RoPE, Attention, SwiGLU equations) *plus* the structural evolution (MoE, Mamba) you asked for.



# Deep Learning: Transformers & Advanced Architectures

Complete Study Guide (Math + Evolution)

Deep Learning Class Study Guide

December 3, 2025

## 1 Introduction

This document synthesizes concepts from Lectures 18, 19, and 20. It maps theoretical concepts directly to seminal papers, includes rigorous mathematical formulations, and tracks the evolution of the architecture to the state-of-the-art.

## 2 The Foundation: Attention & The Transformer

*Lecture 18 & 19*

The Transformer relies on projecting inputs into specific subspaces to calculate compatibility.

### 2.1 Linear Projections (Matrix Form)

Given an input matrix  $X \in \mathbb{R}^{N \times d_{\text{model}}}$  (where  $N$  is sequence length), we learn three weight matrices  $W^Q, W^K, W^V$  to project  $X$ :

$$Q = XW^Q \in \mathbb{R}^{N \times d_k} \quad (1)$$

$$K = XW^K \in \mathbb{R}^{N \times d_k} \quad (2)$$

$$V = XW^V \in \mathbb{R}^{N \times d_v} \quad (3)$$

### 2.2 Scaled Dot-Product Attention

The attention score  $A$  is a row-stochastic matrix representing the similarity between every pair of tokens.

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) V \quad (4)$$

#### Reference: Attention Is All You Need (Vaswani et al., 2017)

- **Go to:** Section 3.2.
- **Scaling Factor:** The division by  $\sqrt{d_k}$  is crucial. Without it, for large  $d_k$ , the dot products grow large in magnitude, pushing the softmax function into regions where it has vanishing gradients.

### 3 Position Encodings: RoPE (Rotary Position Embedding)

Lecture 20

Standard absolute position encodings add a vector:  $x_{pos} = x_{emb} + p_{pos}$ . RoPE *multiplies* the context vector by a rotation matrix.

#### 3.1 The Rotation Matrix

For a 2-dimensional subspace, the rotation of a vector  $\vec{x}$  by position  $m$  is defined as:

$$f(\vec{x}, m) = R_{\Theta, m} \vec{x} = \begin{pmatrix} \cos m\theta & -\sin m\theta \\ \sin m\theta & \cos m\theta \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \end{pmatrix} \quad (5)$$

#### 3.2 Relative Position Property

The core mathematical benefit is that the dot product between two rotated vectors depends **only** on their relative distance  $(m - n)$ , not their absolute positions.

$$\langle f_q(\vec{q}, m), f_k(\vec{k}, n) \rangle = \vec{q}^T R_m^T R_n \vec{k} = \vec{q}^T R_{n-m} \vec{k} \quad (6)$$

**Reference: RoFormer (Su et al., 2021)**

- **Go to:** Section 3.2.1.
- **Key Insight:**  $R_m^T R_n = R_{n-m}$ . This proves translation invariance.

### 4 Modern Components: Gated Architectures

Lecture 20

Modern LLMs (Llama, PaLM) replace the standard ReLU FFN with **SwiGLU**.

#### 4.1 GLU and SwiGLU Equations

A Gated Linear Unit (GLU) controls information flow via element-wise multiplication ( $\odot$ ).

**Standard GLU:**

$$\text{GLU}(x) = (xW + b) \odot \sigma(xV + c) \quad (7)$$

**SwiGLU (State of the Art):** Replaces sigmoid with the Swish activation function ( $\text{Swish}(x) = x \cdot \sigma(x)$ ).

$$\text{SwiGLU}(x) = (xW + b) \odot \text{Swish}(xV + c) \quad (8)$$

**Reference: GLU Variants Improve Transformer (Shazeer, 2020)**

- **Lecture Ref:** Lecture 20, Page 10.
- **Why?** Learnable gating parameters allow the network to select which features are preserved, improving compute efficiency.

## 5 Normalization: RMSNorm

### Lecture 19

Modern architectures use Root Mean Square Normalization (RMSNorm), which assumes mean zero and saves computation compared to LayerNorm.

$$\text{RMSNorm}(x) = \frac{x}{\sqrt{\frac{1}{d} \sum_{i=1}^d x_i^2 + \epsilon}} \cdot \gamma \quad (9)$$

**Reference: Lecture 19, Page 5**

Notes mention input normalization:  $\text{RMS-norm} = \frac{1}{\sqrt{32,000}}$ .

## 6 Efficiency: Mixture of Experts (MoE)

### Lecture 20

To scale models to trillions of parameters without exploding inference cost, we use MoE.

### 6.1 Routing & Sparsity

Instead of one dense FFN, we have  $N$  experts. A "Router" network selects top- $k$  experts for each token.

- **Total Params:** Massive (e.g., 7B).
- **Active Params:** Small (e.g., 1B per token).

**Reference: Switch Transformers (Fedus et al., 2021) & OLMoE (Muennighoff et al., 2024)**

- **Lecture Ref:** Lecture 20 diagrams showing "Switching FFN Layer".
- **Key Concept:** Conditional computation. Only a fraction of the network activates for any given input.

## 7 Further Work: Linear-Time Sequence Modeling

### Lecture 20

The standard Transformer has quadratic complexity  $O(N^2)$  due to the attention matrix  $QK^T$ . New architectures aim for  $O(N)$  scaling.

### 7.1 Mamba & State Space Models (SSM)

Mamba uses selective state spaces to compress context into a fixed-size memory, similar to an RNN but more capable.

- **Transformer:** Keeps full history. Perfect recall, slow for long context.
- **Mamba:** Compresses history. Fast (Linear time), efficient inference.

- **Context:** Lecture 20, Page 10 (Diagrams of H3 and Mamba).

Not really statisfied. The  
“Further Work” section is  
so much weaker than  
before

顯示思路



顯示思路



# Deep Learning: Transformers & Advanced Architectures

Complete Study Guide (Math + Evolution)

Deep Learning Class Study Guide

December 3, 2025

## 1 Introduction

This document synthesizes concepts from Lectures 18, 19, and 20. It maps theoretical concepts directly to seminal papers, includes rigorous mathematical formulations, and tracks the evolution of the architecture to the state-of-the-art.

## 2 The Foundation: Attention & The Transformer

*Lecture 18 & 19*

The Transformer relies on projecting inputs into specific subspaces to calculate compatibility.

### 2.1 Linear Projections (Matrix Form)

Given an input matrix  $X \in \mathbb{R}^{N \times d_{\text{model}}}$  (where  $N$  is sequence length), we learn three weight matrices  $W^Q, W^K, W^V$  to project  $X$ :

$$Q = XW^Q \in \mathbb{R}^{N \times d_k} \quad (1)$$

$$K = XW^K \in \mathbb{R}^{N \times d_k} \quad (2)$$

$$V = XW^V \in \mathbb{R}^{N \times d_v} \quad (3)$$

### 2.2 Scaled Dot-Product Attention

The attention score  $A$  is a row-stochastic matrix representing the similarity between every pair of tokens.

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) V \quad (4)$$

#### Reference: Attention Is All You Need (Vaswani et al., 2017)

- **Go to:** Section 3.2.
- **Scaling Factor:** The division by  $\sqrt{d_k}$  is crucial. Without it, for large  $d_k$ , the dot products grow large in magnitude, pushing the softmax function into regions where it has vanishing gradients.

### 3 Position Encodings: RoPE (Rotary Position Embedding)

Lecture 20

Standard absolute position encodings add a vector:  $x_{pos} = x_{emb} + p_{pos}$ . RoPE *multiplies* the context vector by a rotation matrix.

#### 3.1 The Rotation Matrix

For a 2-dimensional subspace, the rotation of a vector  $\vec{x}$  by position  $m$  is defined as:

$$f(\vec{x}, m) = R_{\Theta, m} \vec{x} = \begin{pmatrix} \cos m\theta & -\sin m\theta \\ \sin m\theta & \cos m\theta \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \end{pmatrix} \quad (5)$$

#### 3.2 Relative Position Property

The core mathematical benefit is that the dot product between two rotated vectors depends **only** on their relative distance  $(m - n)$ , not their absolute positions.

$$\langle f_q(\vec{q}, m), f_k(\vec{k}, n) \rangle = \vec{q}^T R_m^T R_n \vec{k} = \vec{q}^T R_{n-m} \vec{k} \quad (6)$$

**Reference: RoFormer (Su et al., 2021)**

- **Go to:** Section 3.2.1.
- **Key Insight:**  $R_m^T R_n = R_{n-m}$ . This proves translation invariance.

### 4 Modern Components: Gated Architectures

Lecture 20

Modern LLMs (Llama, PaLM) replace the standard ReLU FFN with **SwiGLU**.

#### 4.1 GLU and SwiGLU Equations

A Gated Linear Unit (GLU) controls information flow via element-wise multiplication ( $\odot$ ).

**Standard GLU:**

$$\text{GLU}(x) = (xW + b) \odot \sigma(xV + c) \quad (7)$$

**SwiGLU (State of the Art):** Replaces sigmoid with the Swish activation function ( $\text{Swish}(x) = x \cdot \sigma(x)$ ).

$$\text{SwiGLU}(x) = (xW + b) \odot \text{Swish}(xV + c) \quad (8)$$

**Reference: GLU Variants Improve Transformer (Shazeer, 2020)**

- **Lecture Ref:** Lecture 20, Page 10.
- **Why?** Learnable gating parameters allow the network to select which features are preserved, improving compute efficiency.

## 5 Normalization: RMSNorm

### Lecture 19

Modern architectures use Root Mean Square Normalization (RMSNorm), which assumes mean zero and saves computation compared to LayerNorm.

$$\text{RMSNorm}(x) = \frac{x}{\sqrt{\frac{1}{d} \sum_{i=1}^d x_i^2 + \epsilon}} \cdot \gamma \quad (9)$$

Reference: Lecture 19, Page 5

Notes mention input normalization:  $\text{RMS-norm} = \frac{1}{\sqrt{32,000}}$ .

Doesnt make sense.  
RMS-norm is not a constant.

## 6 Efficiency: Mixture of Experts (MoE)

### Lecture 20

To scale models to trillions of parameters without exploding inference cost, we use MoE.

### 6.1 Routing & Sparsity

Instead of one dense FFN, we have  $N$  experts. A "Router" network selects top- $k$  experts for each token.

- **Total Params:** Massive (e.g., 7B).
- **Active Params:** Small (e.g., 1B per token).

Reference: Switch Transformers (Fedus et al., 2021) & OLMoE (Muennighoff et al., 2024)

- **Lecture Ref:** Lecture 20 diagrams showing "Switching FFN Layer".
- **Key Concept:** Conditional computation. Only a fraction of the network activates for any given input.

## 7 Evolutionary Timeline: 2017 to State-of-the-Art

Mapping the progression of ideas discussed in Lectures 18, 19 & 20.

### 2017: The Transformer (Parallelization)

Paper: Attention Is All You Need (Vaswani et al.)

**Evolution:** Moved from Recurrent Neural Networks (RNNs) to Attention. **Problem Solved:** RNNs process tokens sequentially ( $t_1 \rightarrow t_2 \rightarrow \dots$ ), preventing parallel training. Transformers process all tokens at once ( $O(1)$  sequential operations), allowing massive scaling on GPUs.

### 2018: BERT (Bidirectional Context)

Paper: BERT: Pre-training of Deep Bidirectional Transformers (Devlin et al.)

**Evolution:** Introduced the "Encoder-Only" architecture. **Problem Solved:** Previous models (GPT-1) read left-to-right. BERT reads the entire sentence at once using Masked Language Modeling, enabling deep understanding tasks like classification and Q&A.



### 2020: GPT-3 (Scaling Laws)

*Paper: Language Models are Few-Shot Learners (Brown et al.)*

**Evolution:** Scaled the "Decoder-Only" architecture to 175B parameters. **Problem Solved:** Proved that simply scaling up parameters and data allows a model to solve tasks (translation, summarization) via "prompting" without any gradient updates (fine-tuning).

### 2020: SwiGLU Activation (Refinement)

*Paper: GLU Variants Improve Transformer (Shazeer)*

**Evolution:** Replaced ReLU/GELU with Swish-Gated Linear Units. **Problem Solved:** Standard ReLU ( $\max(0, x)$ ) is crude. Gating mechanisms (multiplying by a sigmoid-like gate) allow the network to control information flow more granularly, yielding better performance-per-flop.

### 2021: RoPE (Relative Position)

*Paper: RoFormer (Su et al.)*

**Evolution:** Replaced absolute position addition with rotary matrix multiplication. **Problem Solved:** Absolute encodings don't generalize to sequence lengths longer than training. RoPE relies on relative angles, allowing models to extrapolate to much longer contexts (e.g., Llama context extension).

### 2021: Switch Transformer (Sparsity)

*Paper: Switch Transformers (Fedus et al.)*

**Evolution:** Replaced the dense FFN with Mixture of Experts (MoE). **Problem Solved:** Increasing parameters to Trillions makes inference too slow. MoE decouples "Parameter Count" (Memory) from "Compute Cost" (FLOPs) by activating only a small subset of weights per token.

### 2023: Mamba (Linear Complexity)

*Paper: Mamba: Linear-Time Sequence Modeling (Gu & Dao)*

**Evolution:** Replaced Attention with Selective State Space Models (SSMs). **Problem Solved:** Attention is  $O(N^2)$  (Quadratic). As context grows (100k+ tokens), it becomes prohibitively slow. Mamba achieves  $O(N)$  (Linear) scaling, making it feasible to process entire books or DNA sequences efficiently.

### 2024: Modern MoE (Fine-Grained Experts)

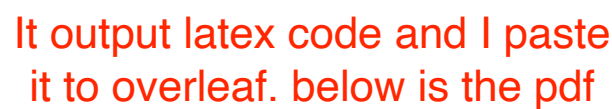
*Paper: OLMoE / DeepSeek V3 (Lecture 20 notes)*

**Evolution:** Evolution of the Switch Transformer. **Problem Solved:** Early MoEs (Switch) routed to 1 large expert, causing instability. Modern MoEs (like DeepSeek/Qwen in Lecture 20) use many small experts (e.g., 64 experts, activate 8), providing smoother training and better specialization.

"Further Work" section is  
much more detailed now  
and have a timeline.  
The depth of the study  
guide is also what I want.

ready

Didn't really include the whole prompt. The whole prompt is just previous latex code + "is there any mistakes of this guide? Fix only the ones that have mistakes. For the parts that are fine, dont change it. Also please fix the latex compilation error. Note that I want to paset this to overleaf and immediately able to compile"



# Deep Learning: Transformers & Advanced Architectures

## Complete Study Guide (Math + Evolution)

Deep Learning Class Study Guide

December 3, 2025

## 1 Introduction

This document synthesizes concepts from Lectures 18, 19, and 20. It maps theoretical concepts directly to seminal papers, includes rigorous mathematical formulations, and tracks the evolution of the architecture to the state-of-the-art.

## 2 The Foundation: Attention & The Transformer

Lecture 18 & 19

The Transformer relies on projecting inputs into specific subspaces to calculate compatibility.

### 2.1 Linear Projections (Matrix Form)

Given an input matrix  $X \in \mathbb{R}^{N \times d_{model}}$  (where  $N$  is sequence length), we learn three weight matrices  $W^Q, W^K, W^V$  to project  $X$ :

$$Q = XW^Q \in \mathbb{R}^{N \times d_k} \quad (1)$$

$$K = XW^K \in \mathbb{R}^{N \times d_k} \quad (2)$$

$$V = XW^V \in \mathbb{R}^{N \times d_v} \quad (3)$$

### 2.2 Scaled Dot-Product Attention

The attention score  $A$  is a row-stochastic matrix representing the similarity between every pair of tokens.

$$\text{Attention}(Q, K, V) = \text{softmax} \left( \frac{QK^T}{\sqrt{d_k}} \right) V \quad (4)$$

**Reference: Attention Is All You Need (Vaswani et al., 2017)**

- **Go to:** Section 3.2.
- **Scaling Factor:** The division by  $\sqrt{d_k}$  is crucial. Without it, for large  $d_k$ , the dot products grow large in magnitude, pushing the softmax function into regions where it has vanishing gradients.

## 3 Position Encodings: RoPE (Rotary Position Embedding)

Lecture 20

Standard absolute position encodings add a vector:  $x_{pos} = x_{emb} + p_{pos}$ . RoPE *multiplies* the context vector by a rotation matrix.

### 3.1 The Rotation Matrix

For a 2-dimensional subspace, the rotation of a vector  $\vec{x}$  by position  $m$  is defined as:

$$f(\vec{x}, m) = R_{\Theta, m} \vec{x} = \begin{pmatrix} \cos m\theta & -\sin m\theta \\ \sin m\theta & \cos m\theta \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \end{pmatrix} \quad (5)$$

### 3.2 Relative Position Property

The core mathematical benefit is that the dot product between two rotated vectors depends **only** on their relative distance  $(m - n)$ , not their absolute positions.

$$\langle f_q(\vec{q}, m), f_k(\vec{k}, n) \rangle = \vec{q}^T R_m^T R_n \vec{k} = \vec{q}^T R_{n-m} \vec{k} \quad (6)$$

**Reference: RoFormer (Su et al., 2021)**

- **Go to:** Section 3.2.1.
- **Key Insight:**  $R_m^T R_n = R_{n-m}$ . This proves translation invariance.

## 4 Modern Components: Gated Architectures

Lecture 20

Modern LLMs (Llama, PaLM) replace the standard ReLU FFN with **SwiGLU**.

### 4.1 GLU and SwiGLU Equations

A Gated Linear Unit (GLU) controls information flow via element-wise multiplication ( $\odot$ ).

**Standard GLU:**

$$\text{GLU}(x) = (xW + b) \odot \sigma(xV + c) \quad (7)$$

**SwiGLU (State of the Art):** Replaces sigmoid with the Swish activation function ( $\text{Swish}(x) = x \cdot \sigma(x)$ ).

$$\text{SwiGLU}(x) = (xW + b) \odot \text{Swish}(xV + c) \quad (8)$$

**Reference: GLU Variants Improve Transformer (Shazeer, 2020)**

- **Lecture Ref:** Lecture 20, Page 10.
- **Why?** Learnable gating parameters allow the network to select which features are preserved, improving compute efficiency.

## 5 Normalization: RMSNorm

Lecture 19

Modern architectures use Root Mean Square Normalization (RMSNorm), which assumes mean zero and saves computation compared to LayerNorm.

$$\text{RMSNorm}(x) = \frac{x}{\sqrt{\frac{1}{d} \sum_{i=1}^d x_i^2 + \epsilon}} \cdot \gamma \quad (9)$$

**Reference: Lecture 19, Page 5**

Notes mention input normalization example:  $\text{RMS-norm} \approx \frac{1}{\sqrt{32,000}}$  (Likely referring to vocabulary size scaling or a specific dimension constant).

Nice it mentions that it is just  
an example

## 6 Efficiency: Mixture of Experts (MoE)

Lecture 20

To scale models to trillions of parameters without exploding inference cost, we use MoE.

### 6.1 Routing & Sparsity

Instead of one dense FFN, we have  $N$  experts. A "Router" network selects top- $k$  experts for each token.

- **Total Params:** Massive (e.g., 7B).
- **Active Params:** Small (e.g., 1B per token).

Reference: Switch Transformers (Fedus et al., 2021) & OLMoE (Muennighoff et al., 2024)

- **Lecture Ref:** Lecture 20 diagrams showing "Switching FFN Layer".
- **Key Concept:** Conditional computation. Only a fraction of the network activates for any given input.

## 7 Evolutionary Timeline: 2017 to State-of-the-Art

Mapping the progression of ideas discussed in Lectures 18, 19 & 20.

### 2017: The Transformer (Parallelization)

Paper: *Attention Is All You Need* (Vaswani et al.)

**Evolution:** Moved from Recurrent Neural Networks (RNNs) to Attention. **Problem Solved:** RNNs process tokens sequentially ( $t_1 \rightarrow t_2 \rightarrow \dots$ ), preventing parallel training. Transformers process all tokens at once ( $O(1)$  sequential operations), allowing massive scaling on GPUs.

### 2018: BERT (Bidirectional Context)

Paper: *BERT: Pre-training of Deep Bidirectional Transformers* (Devlin et al.)

**Evolution:** Introduced the "Encoder-Only" architecture. **Problem Solved:** Previous models (GPT-1) read left-to-right. BERT reads the entire sentence at once using Masked Language Modeling, enabling deep understanding tasks like classification and Q&A.

### 2020: GPT-3 (Scaling Laws)

Paper: *Language Models are Few-Shot Learners* (Brown et al.)

**Evolution:** Scaled the "Decoder-Only" architecture to 175B parameters. **Problem Solved:** Proved that simply scaling up parameters and data allows a model to solve tasks (translation, summarization) via "prompting" without any gradient updates (fine-tuning).

### 2020: SwiGLU Activation (Refinement)

Paper: *GLU Variants Improve Transformer* (Shazeer)

**Evolution:** Replaced ReLU/GELU with Swish-Gated Linear Units. **Problem Solved:** Standard ReLU ( $\max(0, x)$ ) is crude. Gating mechanisms (multiplying by a sigmoid-like gate) allow the network to control information flow more granularly, yielding better performance-per-flop.

### 2021: RoPE (Relative Position)

Paper: *RoFormer* (Su et al.)

**Evolution:** Replaced absolute position addition with rotary matrix multiplication. **Problem Solved:** Absolute encodings don't generalize to sequence lengths longer than training. RoPE relies on relative angles, allowing models to extrapolate to much longer contexts (e.g., Llama context extension).

### 2021: Switch Transformer (Sparsity)

Paper: *Switch Transformers* (Fedus et al.)

**Evolution:** Replaced the dense FFN with Mixture of Experts (MoE). **Problem Solved:** Increasing parameters to Trillions makes inference too slow. MoE decouples "Parameter Count" (Memory) from "Compute Cost" (FLOPs) by activating only a small subset of weights per token.

**2023: Mamba (Linear Complexity)**

*Paper: Mamba: Linear-Time Sequence Modeling (Gu & Dao)*

**Evolution:** Replaced Attention with Selective State Space Models (SSMs). **Problem Solved:** Attention is  $O(N^2)$  (Quadratic). As context grows (100k+ tokens), it becomes prohibitively slow. Mamba achieves  $O(N)$  (Linear) scaling, making it feasible to process entire books or DNA sequences efficiently.

**2024: Modern MoE (Fine-Grained Experts)**

*Paper: OLMoE / DeepSeek V3 (Lecture 20 notes)*

**Evolution:** Evolution of the Switch Transformer. **Problem Solved:** Early MoEs (Switch) routed to 1 large expert, causing instability. Modern MoEs (like DeepSeek/Qwen in Lecture 20) use many small experts (e.g., 64 experts, activate 8), providing smoother training and better specialization.